COMPLETE SYSTEM DOCUMENTATION

Nexora

The Next Way to Work Together

Scope, functionality and implementation status

A shared delivery workspace in which an agency plans and executes its work and the customer watches that work happen — on the same tickets, at the same time, but seeing a deliberately different slice of it.

REPOSITORY	COMMIT	STATUS	LICENCE
aqueduct/workspace	30bfc83	Pre-1.0, active	Proprietary
COMPILED			
17 Sep 2026			

HOW TO READ THIS DOCUMENT

Every statement here was derived from the source at the commit above — routes, migrations, policies, services, container configuration and the test suite. Nothing describes a feature that does not exist. Where a capability is present but incomplete it is marked Partial; where it is designed for but absent, Not built. Section 15 collects every such marking in one table.

Contents

01 Executive summary

02 The customer boundary

03 Roles and permissions

04 Boards and tickets

05 Documentation workspace

06 The workspace AI

07 AI ticket runs

08 Integrations

09 Reporting and realtime

10 Architecture

11 Security

12 Data model

13 Deployment and operations

14 Quality assurance

15 Implementation status

16 Findings and next steps

17 Glossary

Every entry is a live link. Section 15 collects the implementation status of every capability discussed; section 16 lists the findings raised while compiling this document.

01

Executive summary

Nexora replaces four tools with one, and earns the right to do so by solving the problem that stops agencies sharing a board with their customers in the first place.

355 PHP classes	**35** Livewire screens	**41** Migrations	**27** Domain enums	**139** Test files
1,718 Passing tests				

What it is

Nexora combines four things that are normally four separate products: **Kanban boards and tickets** for the work itself; a **documentation workspace** with a nested page tree and a live editor; an **AI assistant with real access to the workspace** that reads through the asking person's own permissions and can draft changes for a human to confirm; and **integrations** with the tools engineering already lives in — GitHub, Slack and SMS.

It is a single Laravel application, server-rendered, deployed as one Docker image run three ways. There is no separate front-end application, no API gateway and no microservices.

The problem it solves

PROBLEM ONE

Customers cannot see progress without a person exporting it

Status meetings exist because nobody outside the team can read the board. Nexora gives the customer a real login to the real board — and answers the obvious objection by making *internal* the default for everything, with publication a deliberate, audited, staff-only act.

PROBLEM TWO

Internal candour and customer communication collide

A team that shares its board stops writing frankly on it. Nexora gives every ticket two conversation streams — one the customer takes part in, one the customer cannot see, cannot count and cannot detect — so the team keeps its working notes without keeping a second system.

PROBLEM THREE

Knowing what is happening costs time

Somebody has to read forty tickets to answer “what is overdue?”, or open GitHub to answer “did that ship?”. Nexora’s assistant answers those from the live workspace, using retrieval tools scoped to the asker, and will draft the ticket that should follow.

Who uses it

AUDIENCE	ROLE	WHAT THEY ARE THERE FOR
Agency administrators	`admin`	Creating boards and accounts, holding AI credentials, setting how far automation is trusted, seeing every board
Delivery team members	`team`	Daily work: tickets, documentation, the assistant, integrations, and board configuration for boards they belong to
Customer stakeholders	`customer`	Raising requests, following progress, reading published documentation, asking a read-only assistant about their own project

There is no self-service sign-up by default. `ALLOW_PUBLIC_REGISTRATION` is `false` and an administrator creates accounts. When enabled, a self-registered account receives the least-privileged role, `customer` — never staff.

Capabilities at a glance

CAPABILITY	STATUS
Kanban boards with drag-and-drop columns and cards	Built
Tickets: type, priority, assignee, estimate, due date, labels, checklist, links	Built
Two-stream ticket conversation (customer / internal)	Built
Nested documentation workspace, WYSIWYG editing, autosave drafts	Built
Workspace AI assistant — panel and full page — with ten retrieval tools	Built
AI proposals: create and update tickets and pages, comment, delete, start a run	Built
AI-generated charts, tables and KPI rows, with CSV and PNG export	Built
Spoken conversation (speech-to-text and text-to-speech)	Built
AI file attachments: text, Markdown, CSV, PDF, images, audio, Office documents	Built
GitHub inbound webhooks: branches, commits, pull requests, check suites, statuses	Built
GitHub outbound: push a branch, open a draft pull request	Built
Slack notifications, per board and per event; SMS alerts for critical tickets	Built
Statistics: team and customer screens, flow metrics, CSV and PNG export	Built
Realtime board, conversation and notification updates	Built
Command palette global search; workspace activity feed; light/dark/system theme	Built
AI ticket runs — *suggest* mode, end to end	Built
AI ticket runs — *apply* mode (opens a draft pull request)	Partial
Live external knowledge lookup for the assistant	Partial
AI Standup Generator	Not built
Remote MCP server for Claude / ChatGPT (reads and changes, OAuth sign-in, same permissions as the app)	Built
General REST / JSON API for third parties	Not built

02

The customer boundary

One idea runs through every feature in this product, and it is the reason the rest of it can exist: a customer may never observe internal content, on any board, including boards they belong to.

	CUSTOMER SEES	STAYS INTERNAL
Tickets	Tickets marked customer-visible, on boards they are a member of	Every internal ticket — and its count, and that it exists at all
Conversation	The customer stream on those tickets, and their own replies	The internal stream: notes, their authors, their number
Documentation	Published pages whose every ancestor is also published	Internal pages, and the whole tree standing above them
Code activity	Nothing	Branches, commits, pull requests, CI status, attached repositories
AI	A read-only assistant over their own material	AI runs, their cost, their failures, and the full-page board chat
Reporting	Six aggregate metrics, counted over tickets they can open	Assignee workload, cycle time, AI spend, the internal/visible split
History	Nothing	The whole workspace activity feed

A denial is **404, never 403**, wherever existence is itself a secret — a 403 on `/boards/acme-platform` would confirm that such a board exists.

Where the rule is enforced

Four independent layers, designed to overlap. Authorization is applied in SQL first and in policies second: a collection is filtered by a query scope, and a single loaded record is then checked by the matching policy method. Each policy is written to mirror its scope line for line.

01 **Route middleware** — `role:admin,team` blocks before a component exists; `password.confirm` re-authenticates for administration screens. Coarse only.

02	**Query scopes**	**The primary defence.** `Ticket::visibleTo()`, `Comment::visibleTo()`, `AiRun::visibleTo()` and `DocPageFinder` filter in SQL — which protects lists, search, counts and links at once.
03	**Policies**	Eight of them. A single loaded record is checked against the scope it came from. Attachments hold no rules of their own — they resolve their owner and ask *that* record's policy.
04	**Actions**	Field-level rules that hold whatever calls them — a screen, a console command, or a confirmed AI proposal. There is no second, weaker path into the same tables.

THE DESIGN DECISION THAT CARRIES THE MOST WEIGHT

`customer_visible` is stored **positively and defaults to** `false` on both tickets and documentation pages, so a row created by code that forgets to set it is internal rather than exposed. It is deliberately **not mass-assignable**: only the ticket actions can set it. The same applies to a comment's stream — staff choose explicitly, and the default when nothing is said is *internal*, so a code path that forgets to choose keeps quiet rather than publishing to a client.

WHY CUSTOMERS GET A NARROWER KIND OF REFUSAL

A customer's restricted fields are **silently dropped** rather than rejected. The ticket policy permits a customer to edit a ticket they raised; the *action* is what narrows which fields they may touch. So a crafted request that submits `assignee_id`, `estimate` or `customer_visible` cannot set any of them, even though the update itself is authorized.

03

Roles and permissions

Three application-wide roles answer “what kind of person is this?”. Board membership answers “what may they do on board X?”. The two are separate, and that separation is deliberate.

ROLE	VALUE	SEES EVERY BOARD	MAY SEE INTERNAL CONTENT	MAY ADMINISTER
Administrator	`admin`	**Yes**, without membership	**Yes**	**Yes**
Team member	`team`	Memberships only	**Yes**	No
Customer	`customer`	Memberships only	**Never**	No

The enum groups the first two as *staff*. The internal-content rule is written as a deny-list of one, so that any future role is internal-unsafe unless it is deliberately added to the staff set.

Workspace and boards

CAPABILITY	ADMINISTRATOR	TEAM	CUSTOMER
See the board index	**All boards**	Own memberships	Own memberships
Open a board	**Any**	Member only	Member only
Create, rename or delete a board	**Yes** + password confirm	No	No
Add or remove board members	**Yes**	No	No
Create, rename, reorder, delete columns and labels	**Yes**	**Member only**	No
Configure board AI settings, repositories, Slack and SMS	**Yes**	**Member only**	**404**
Global AI settings `/admin/ai`	**Yes** + confirm	403	403
Create, edit, deactivate users; change another user’s role	**Yes** + confirm	403	403
Change **own** role, or delete **own** account	**Forbidden**	No	No
Edit own profile, password and theme	**Yes**	**Yes**	**Yes**

The split is deliberate. The board *record* — its name, prefix, membership and existence — stays administrator-only. The board’s *working surface* is the delivery team’s daily work on their own board.

Tickets

CAPABILITY	ADMINISTRATOR	TEAM	CUSTOMER
See an internal ticket	**Yes**	Member only	**Never**
Create a ticket	**Yes**	**Yes**	**Yes**, constrained
Choose the column at creation	**Yes**	**Yes**	**Always first column**
Set visibility at creation	Defaults customer-visible	Defaults customer-visible	**Always customer-visible**
Assign at creation	**Yes**	**Yes**	**Always unassigned**
Edit a ticket raised by somebody else	**Yes**	**Yes**	No
Change assignee, estimate or customer visibility	**Yes**	**Yes**	**Silently dropped**
Move between columns; add labels; link tickets	**Yes**	**Yes**	No
Manage the checklist; attach files	**Yes**	**Yes**	Own tickets only
Delete a ticket	**Yes**	**Yes**	No

Conversations, documentation and files

CAPABILITY	ADMINISTRATOR	TEAM	CUSTOMER
Read and post to the customer conversation	**Yes**	**Yes**	If the board allows it
Read internal notes	**Yes**	**Yes**	**Never — not even the count**
Edit a comment	Own only	Own only	Own only
Delete a comment	**Own, and anybody's**	Own only	Own only
Read published documentation	**Yes**	**Yes**	Only if every ancestor is published
Read internal documentation	**Yes**	Member only	**Never**
Create, edit, move, delete or publish a page	**Yes**	**Member only**	No
Download an attachment	If the owning record is readable — a file has no visibility of its own		

AI

CAPABILITY	ADMINISTRATOR	TEAM	CUSTOMER
Open the assistant panel	**Yes**	**Yes**	**Yes** — read-only
Open the full-page board AI chat	Member boards	Member only	**404**
Read somebody else's conversation	**No — ownership is absolute**	No	No
Use the voice interface	**Yes**	**Yes**	**Yes**
Be offered *write* tools by the model	Mode permitting	Mode permitting	**Never, in any mode**
Confirm an AI proposal	Mode permitting	Mode permitting	No
See, start or cancel AI runs on a ticket	**Yes**	Member only	**404**
Set the workspace capability mode	**Yes**	No	No

A TOOL A PERSON MAY NOT USE IS NOT SENT TO THE MODEL AT ALL

Rather than sent and then refused. A model with no write tool cannot produce a proposal, so there is no preview, no confirm button and nothing for a crafted request to execute. An unrecognised tool name and a tool that exists but is withheld produce the *same* refusal text, so a customer cannot tell the two apart.

04

Boards and tickets

A board is the unit of work, the unit of access and the unit of configuration. Access to Nexora is access to a set of boards; there is no entity above one.

The board

A board carries a globally unique **slug** used in every URL and a globally unique **ticket prefix** of two to six characters. Primary keys are never exposed. On creation it receives one tab, *Main*, with the standard workflow — Backlog, To Do, In Progress, Review, Done — with *Done* marked terminal.

Board options live in a JSON `settings` column read through a helper that falls back to configuration defaults, so a new board option needs no migration and no backfill. Columns are unique per tab and reorderable; deleting one moves its tickets out one at a time and deliberately **suppresses the "moved to" notification**, because otherwise tidying a board would notify everyone assigned a card on it.

Tabs

A board can be split into **tabs** — "Admin dashboard", "Public website" — like browser tabs: one board, several boards' worth of work. Each tab has its own columns; the strip above the columns switches between them without leaving the board, keeps the filters on, and shows each tab's count of matching tickets. A ticket has no tab of its own: **its tab is its column's tab**, enforced by a composite foreign key, so a ticket can never sit in another tab's column.

The delivery team adds, renames, reorders and removes tabs from the strip itself. A new tab copies the open tab's columns; deleting a tab never deletes its tickets — they move to the tab chosen, by column name, then done to done, then the first column, and the panel shows where each will go first. A ticket moves between tabs by dragging its card onto a tab, from the ticket page, or through the assistant and MCP, which take an optional `tab`. Customers see every tab's name, as they see columns, but only their own tickets and counts.

Anatomy of a ticket

FIELD	TYPE	NOTES
number	`integer`	Per board, allocated under a row lock. Unique with `board_id`. The human key is `PREFIX-NUMBER` — `NL-42`
title	`string(200)`	Required
type	`enum`	`bug`, `task`, `feature` — stored and filterable; no workflow differs by it
description_md	`text`	Markdown, written through the rich-text editor
priority	`enum`	`critical`, `high`, `medium`, `low`, `nice_to_have`
assignee_id	`FK users`	Active, non-customer members of the board only
estimate	`decimal(6,2)`	Staff-only field
due_date	`date`	Indexed
customer_visible	`boolean`	**Defaults to false.** The customer boundary
board_column_id	`FK columns`	Status. Restricted on delete — a column cannot be removed from under its tickets

Numbers are allocated inside the create action with `SELECT … FOR UPDATE` on the board row.
Combined with the unique index, two simultaneous requests cannot receive the same number.

The two conversation streams

	CUSTOMER STREAM	INTERNAL STREAM
Who reads it	Everyone who can read the ticket	Staff only
Who writes to it	Staff always; customers if the board allows customer comments	Staff only
The default	A comment written by a customer is always here, whatever the request said	Where a staff comment lands when nothing is chosen

Everything that follows a post — who is notified, what is broadcast — is derived from the stream that was actually stored, never from what the caller asked for.

What a ticket carries besides prose

TICKET_SUBTASKS

Checklist

Title, completion, position, who completed it and when. Deliberately not written to the timeline — a checklist changes often and would drown the events that matter. Convertible into Markdown checkboxes on a button press, preserving the metadata a checkbox cannot express.

TICKET_LINKS

Links

`blocks` and `relates_to`, staff-only, and only to tickets the person can already see.

TICKET_LABEL

Labels

Board-scoped, unique per board, from a fixed colour palette. A customer sees the labels on a ticket they may read but cannot change them.

TICKET_EVENTS

Append-only history

Twenty event types spanning human actions, AI runs and GitHub activity, in one unified timeline — so “branch created” sits between “Vikas created NL-3” and “Alex changed priority”, not in a separate panel.

What happens when a ticket changes

One observer fans every change out, because “every change to a ticket is announced” is a property of the model rather than of any one screen.

BROADCAST	NOTIFY	AI	SLACK	SMS
An internal ticket signals only the internal channel, so a customer’s browser is never even woken	One row per recipient, each policy-checked against the thing being notified about	On creation only, and only for a customer’s ticket — subject to board settings, mode and the daily cap	Per board, per event, opt-in. Queued	Critical tickets only. Per board opt-in. Queued

None of these may throw — nothing here is important enough to fail a ticket save. Position rewrites during a drag go through the query builder rather than through models, so shuffling ten cards produces **one** announcement from the ticket that actually moved, not eleven.

05

Documentation workspace

Each board carries its own nested page tree with a live editor, per-page publication to customers, attachments and a contents list — where runbooks, decisions and customer-facing guides live next to the work they describe.

Structure

PROPERTY	DETAIL
Layout	Three columns: page tree, document, and an “On this page” contents list
Parent	Self-referencing, restricted on delete
Maximum depth	**4.** So ancestor walks terminate in a known number of queries, and the sidebar stays legible
Ordering	Drag-and-drop within each parent
Slug	Unique per board, used in the URL — never the primary key
Icon	An optional emoji from a picker

Importing files as pages

Import file beside *+ New page*, **Import a file** in an empty documentation section, and *Import a file inside this one* on any page — and the sidebar and the empty state take dropped files. PDF, Word, text, Markdown, CSV, Excel and PowerPoint files each become an **internal page with the original attached**, keeping the structure a to-do or bug list is made of: headings, nested lists, checkboxes as to-dos, tables, links. A scanned PDF or a legacy `.doc` is refused with what to do; one bad file never stops the rest. Office XML is read with a streaming reader that refuses any document type declaration, inside the existing zip-bomb limits, and large files are converted on the queue.

A page with open to-dos shows **Create tickets with AI**: the assistant opens on that page’s board, in Writing, with the request written — nothing is created until it is confirmed.

The ancestor visibility rule

> **A TREE NEEDS ONE RULE A FLAT TABLE DOES NOT**
>
> A page is visible only when **it and every one of its ancestors** are visible. Publishing a child of an internal page would otherwise show a customer a page whose parent, path and siblings they must not see — and the breadcrumb alone would leak the internal titles.

It is enforced in three deliberately overlapping places: **on write**, publishing is refused when a parent is internal and "make internal" cascades down the whole subtree; **on reading one page**, the finder walks the ancestor chain through the visibility scope and 404s if any link is missing; and **on reading the tree**, any node whose parent is not in the visible set is dropped, so a broken invariant cannot surface as an orphan.

The two operations are deliberately **asymmetric**. Publishing is *refused* rather than silently publishing the ancestors too, because exposing a parent page is the author's decision, not a side effect. Un-publishing *cascades*, because anything else would leave published children hanging off an internal parent. Retracting always fails closed: more is hidden, never less.

The editor is not a mode

Anybody who may edit a page gets the editor **immediately, on the page they opened**. There is no Edit button and no read-only step in front of it: a documentation tool whose default state is "looking at" rather than "working on" makes every small correction a four-click job, which is how documentation goes stale. A reader who may not edit gets the rendered document, and never so much as the markup of a control they could not use.

The editor is TipTap (ProseMirror), registered as an Alpine component and used headless, so the toolbar is ordinary Blade buttons styled like every other button in the product. Supported: headings, bold, italic, underline, strikethrough, inline code, code blocks with syntax highlighting, links, bullet, ordered and task lists, block quotes, horizontal rules, images and tables.

MARKDOWN IS THE STORAGE FORMAT

The WYSIWYG editor is a surface over Markdown, and the HTML-to-Markdown conversion happens **server-side**. The stored artefact is always `body_md`, which keeps documents readable, searchable, diffable and exportable without the editor — and means a future change to how documents are rendered cannot alter what was stored.

Autosave lands by audience

	PUBLISHED PAGE	INTERNAL PAGE
Autosave writes to	`draft_md` — held back	`body_md` — committed immediately
Why	A page a customer is reading changes when its author says so, not between keystrokes	Nobody outside the team can read it, so there is no audience to show half-written prose to
Release	Publish moves the draft into the body, records the edit, tells other clients, discards the draft	None needed

Autosave never broadcasts, on either branch — a colleague reading the board should not have their screen pulled out from under them because somebody two desks away is typing.

Two properties make autosaving on a timer safe: the update action returns early unless the row is actually dirty, and the activity logger coalesces an editing session into one feed entry per person per page. An afternoon's typing is a handful of writes and one line in the feed.

NOT IMPLEMENTED IN THIS MODULE

CAPABILITY	STATUS
Page version history — only the current body and one draft are stored	Not built
Page comments	Not built
Templates; export a page or space to PDF or Word	Not built
Real-time collaborative editing — autosave is single-author	Not built

06

The workspace AI

Nexora's largest subsystem — roughly a third of the application's classes. Almost every question about AI behaviour resolves to one of three settings, and none of them can grant what another refuses.

SETTING	WHO CHOOSES IT	WHAT IT DECIDES
Capability mode	An administrator for the workspace; the team for a board	The **ceiling** on what the AI may ever attempt here
Chat mode	The person asking, per conversation	What this conversation is *for* — reading, writing, or both
Knowledge scope	The person asking, per conversation	Where an answer may be *sourced* from

Capability mode — the security dial

MODE	READ AND ANSWER	PROPOSE WRITES	TICKET ANALYSIS	WRITE CODE	RUN UNATTENDED
AI Observer	Yes	No	No	No	No
AI Operator	Yes	Confirmed by a person	Suggest runs	No	No
AI Agent	Yes	Yes	Suggest and apply	Yes	Yes

Three cumulative levels rather than a matrix of switches, because a matrix reads well on a settings screen and is impossible to reason about in a security review. Observer refuses ticket runs entirely, and that is stricter than people expect on purpose: a suggest run ends by posting an internal note, and a note is a write.

THE CAPABILITY MODE IS A CEILING, NEVER A FLOOR

Every method on the capability guard can only ever *deny*. Raising a workspace to AI Agent gives no user a capability they did not already have by hand. The order of checks on any write is always: (1) may this person use the assistant at all, decided by the route group, board policy and SQL scopes; (2) does the mode permit the AI to attempt this; (3) is **this person** allowed to make **this** change, applied against the confirming user by the ordinary action.

Chat mode and knowledge scope

CHAT MODE

Reading · Writing · Everything

Reading is the default, so the setting somebody gets by not choosing is the one that changes nothing. **Writing** withholds the lookups so that “write me a ticket for the login bug” does not become four searches and a summary before it gets to the draft. Each mode names its own model, because lookup-and-summarise and drafting-something-actionable have different shapes.

KNOWLEDGE SCOPE

Project only · Outside Project

Project only is the default and is not configurable away. Outside is *additive*, never a replacement — the board context still goes in and a question about NL-5 is still answered from NL-5, because “compare our architecture with a typical Laravel SaaS” needs both halves at once.

KNOWLEDGE SCOPE HAS NOTHING TO DO WITH AUTHORIZATION

Switching it does not widen what a person may read by a single row. It is enforced by the **data layer** — the tool registry decides which tools exist for a turn — not by the prompt. One honest limit is written down in the code: a language model’s own trained knowledge is not a data source this application can withhold, so under Project the assistant is *instructed* to decline general questions, and that is a prompt rather than a wall. What the data layer does guarantee is narrower and stronger: no external service is contacted, and no project data is widened either way.

The two surfaces

SURFACE	WHO	NOTES
Assistant panel	Everyone with at least one reachable board, **customers included**	Mounted in the layout, persists across navigation. Costs nothing until opened. **Never aborts** — every failure narrows it to its board picker rather than taking down the page it is rendered on
Board AI chat	**Staff members of the board only** — 404 otherwise	Full page. Quotes internal tickets, internal notes and internal documentation

The two exist as two abilities rather than one loosened ability, because the surfaces differ in what they are allowed to contain and a single relaxed check would have quietly opened the staff page as well.

The ten retrieval tools

TOOL	WHAT IT READS	AVAILABILITY
`get_board`	Description, columns with counts, labels, prefix	Everyone
`get_ticket`	One ticket in full, plus recent conversation and history	Everyone — scoped
`search_tickets`	By text, column, label, priority, assignee, due date, age	Everyone — scoped
`search_documentation`	Page titles and bodies, with extracts	Everyone — scoped
`get_documentation_page`	One page in full, by slug	Everyone — scoped
`get_statistics`	Eight aggregate metrics	Everyone; three metrics staff-only
`get_activity`	Workspace activity history	**Staff only**
`get_code_activity`	Branches, commits, pull requests, CI state	**Staff only**
`get_github_repository`	Attached repositories, default branch, build commands — never credentials, never repository contents	**Staff only**
`search_external_knowledge`	An outside service	Outside Project scope *and* a configured driver

The three statistics metrics withheld from customers are per-assignee workload, per-label counts and the internal/visible split. The five they may ask for are totals, by status, by priority, created by week and completed by week.

FOUR PROMISES MADE STRUCTURAL

SCHEMA

Enforced before entry

Each tool declares its schema, and a call that does not fit is answered with prose the model can act on — not an exception.

AUTHORIZATION

The asker is the viewer

A tool argument names a *subject*; the viewer comes from the *request*. An assistant with tools answers more questions than one without, about exactly the same material.

CONTAINMENT

A tool that throws is contained

The exception is logged in full and reported to the model as “that lookup failed”, with nothing about why — a message can carry a query, a path or a column name.

AUDIT

One row per call, whatever the outcome

Written to `ai_tool_invocations`. A refused call is the interesting one to have a record of.

The loop is bounded at 4 rounds, 6 calls per round and 60,000 characters of tool material per question. Those are cost and latency controls, **not** security controls — turning tools off narrows what the assistant can answer and grants nobody anything.

Write tools: propose, then confirm

Seven action types, each exposed to the model as `propose_<name>`. The prefix is deliberate: the name itself tells the model that calling it does not perform the action. Six are reversible — create and update a ticket, comment on one, create and update a documentation page, start a code run. One is destructive — delete a ticket — and is **always confirmed by typing the ticket's key by hand**, in every mode.

STEP 1
Model emits one proposal. One is kept per turn — a preview showing two unrelated changes invites confirming both while reading one — except new tickets: up to 30 of them are one proposal, previewed as a list and created all or none

STEP 2
Under Operator: a preview and a Confirm button. Under Agent: reversible actions are carried out immediately

STEP 3
The target is re-resolved from the stored proposal, through readers that apply visibility

STEP 4
The capability mode is re-checked — a preview can be on screen when an administrator lowers it

STEP 5
Gate checks run against the **confirming** user, **per field**, all before any write

STEP 6
The ordinary action is called — every rule it owns still applies

WHAT THE SCHEMAS DELIBERATELY CANNOT EXPRESS

Visibility can only be narrowed, and only at creation. A new ticket starts from the board's default (customer-visible unless the board says otherwise) and the assistant may make it internal, never more visible than that. Publishing an existing ticket is not reachable from the chat. There is no field that can name a viewer, a user id, a column id or a label id. Assignees, columns and labels are named *in words* — "Alex Round", "In Progress", "bug" — because a name has to be *resolved*, and that is the step where the resolver can refuse an assignee who is not on the board and ask which of two matching columns was meant.

Authorization is **per field, and happens first.** A request to move a card *and* re-assign it asks both policies, and both are taken before either write happens — so a team member who may move but not assign changes nothing at all and is told why, rather than getting a moved card, a dropped assignee and a vague error.

Sessions, budgets and cost

LIMIT	DEFAULT	VARIABLE
Tokens per session (a spending cap)	`off`	`AI_SESSION_TOKEN_LIMIT`
Tokens per user per day (cached reads count a tenth)	`2,000,000`	`AI_DAILY_USER_TOKEN_LIMIT`
History replayed	`the whole conversation, up to the model's window`	`AI_CHAT_CONTEXT_BUDGET`
Prompt caching	`on, 5 minutes`	`AI_CHAT_CACHE`
Output tokens per answer	`4,000`	`AI_CHAT_MAX_OUTPUT_TOKENS`
Context: one board	`60 tickets, 40 pages, 40 events`	—
Context: whole workspace	`12 boards, 6 tickets each, titles only`	—

> A CONVERSATION BELONGS TO ONE PERSON — AN ADMINISTRATOR INCLUDED
>
> There is no reading right over a colleague's thread, and therefore none over the documents in it. Every denial is a 404. Transcripts persist; **prompts do not** — the system prompt and the context are rebuilt on every request, so editing a board's project context changes the next answer instead of being frozen into an old row.

The whole conversation, cached. Each question replays the conversation in full, up to the model's real context window — a million tokens for Opus 5 and Sonnet 5 — and asks the provider to cache it, so the repeat costs a tenth of a fresh send. A conversation that outgrows the window loses its oldest part in one jump, from a question, and the beginning then stays the same so the cache keeps working. Files stay with the question they came in on. The note above the composer now says when the conversation nearly fills what the model can read at once; the per-conversation spending cap is off by default.

Usage is recorded per exchange with a `usage_reported` flag: a run whose token counts were never reported has **no** cost and is shown as an unpriced run rather than folded in as zero. Pricing lives in exactly one place, and a model that is not listed produces a null cost rather than a guess.

Charts, voice and attachments

VISUALISATION

Seven chart types, drawn server-side

Line, bar, horizontal bar, area, pie, donut and scatter, plus tables and KPI rows. **The model never supplies anything that is executed or rendered directly** — it supplies numbers and labels, which are validated into typed specs, and the server draws the SVG from those. Colours come from a fixed palette chosen by index. Failure is always prose: a malformed chart falls back to the text the model wrote.

EXPORT

CSV by block index, PNG in the browser

The CSV route re-reads the stored answer and re-parses it with the same parser the screen used, then exports block *N* — it addresses an index rather than accepting a payload, because a browser that could POST its rows could export rows the model never produced. PNG is produced from the SVG already on the page.

VOICE

Three steps, not a fourth kind of session

Speech-to-text, the ordinary assistant, text-to-speech — which is what lets a spoken question inherit sessions, attachments, capability modes, the audit trail and the customer boundary without any of them being taught about voice. Generated audio is **bytes, never a file**, and there is deliberately no cache.

ATTACHMENTS

Seven kinds, kept with their message

Text, Markdown, CSV, PDF, image, audio and Office documents. A file goes in whole with the question it came in on and is replayed with it on every later turn, from the cache, so a long to-do list can be finished with "continue". Word files arrive with their headings, lists and checkboxes. **Every truncation is stated in the prompt**, because a model handed half a contract and not told will summarise it as though it read the end.

IMPORT

A file into the docs, and its tickets

"Import this into the docs and make the tickets" is one proposal: `propose_import_document` makes an internal page whose body is the whole file, converted on the server — never retyped by the model — and a ticket for each open to-do and bug, internal unless asked otherwise, all or nothing. A page with open to-dos offers *Create tickets with AI*, which opens the assistant on that page.

CSV rows are sent as data *and* the profile — column types, ranges, outliers — is computed from *all* of them, so a question about an average is answered from every row even when only the first 300 are quoted. Images are re-encoded rather than forwarded, stripping EXIF including GPS, and are sent as pictures only to models that declare vision. An instruction *inside* an uploaded document is framed to the model as text somebody wrote, never a command.

From Claude or ChatGPT (remote MCP)

People add `https://<host>/mcp` to Claude or ChatGPT as a custom connector, sign in with their Nexora account, and then ask questions — or

ask for changes — from there. Reading uses **the same ten tools through the same registry**, so an answer cannot differ from what the in-app assistant would read for the same person. Changes go through **the same code a confirmed proposal does** (`ExecuteChatAction::perform()`), as the person, within the board's AI mode, and every call is audited on the `mcp` channel.

WHO	CAN CHANGE
Staff	**`create_ticket`**, **`create_tickets`** (up to 30, all or nothing), **`update_ticket`** (fields, status, assignee, labels, due date, estimate), **`update_checklist`**, **`add_comment`**, **`delete_ticket`**, **`create_doc_page`**, **`update_doc_page`**
Customers	Only what the ticket screen already allows: raise a ticket, edit the title and description of one they raised, reply on the customer thread

CONFIRM

Three changes wait for a yes

Deleting a ticket, a comment the customer will read and editing a published page answer "confirmation needed" with a one-time code bound to the person, the target and the exact arguments. Everything else happens on the call — the AI app has already asked the person.

SAFE

Retries, races, runaways

An identical request within 90 seconds returns the first answer instead of a duplicate; a ticket changed since it was read is not overwritten; changes are limited to 30 a minute per person; `MCP_WRITES_ENABLED=false` returns to read-only without a deploy.

HONEST

The answer is the saved row

Every reply is read back from the database: what changed, old → new. A refusal says why; a ticket the person cannot see reads exactly like one that does not exist.

TRACED

"via Claude" in the history

The person is always the actor; the ticket history and activity feed add the app it came through.

How it looks in the chat. The server tells the AI app to answer in the chat, never in an artifact: one ticket as a bold linked line with bullets, several as a table *Ticket | Title | Status | Assigned to | Priority | Due*, status and assignee always shown separately. The tools put status colours in their own output — done, in progress, review, blocked, not started — worked out from the real column, with its name kept, plus for the assignee (*Unassigned* when nobody has it) and a link on every ticket key. Connections approved while the server was read-only were signed out once, so everybody agrees to the new screen.

07

AI ticket runs

A run is the AI working on one ticket, off the request cycle, on its own queue. It is a different feature from the assistant: the assistant is a conversation, a run is a job with a result.

MODE	WHAT IT DOES	REQUIRES
Suggest	Reads the ticket and repository metadata, writes an opinion as an **internal note** on the ticket	AI Operator or above
Apply	Works in an isolated clone, on a new branch, runs the configured tests, and opens a **draft pull request**	AI Agent, a GitHub token, an attached repository, and a code-generation driver

The apply pipeline

PREPARING
Check the coding runtime exists; clone into the run's own directory

REFUSE
If the base branch is protected: main, master, production, release, develop

BRANCH
Create `ai/NL-42-<uuid>` off the base

CODING
The runtime edits files; the diff is re-derived from `git status`, not from what it claimed

REFUSE
An empty diff fails the run — no pull request is opened for work that did not happen

TESTING
Run the configured validation commands

REFUSE
A non-zero exit fails the run, **before** anything is pushed

PULL REQUEST
Commit, push the **new branch only**, open a **draft** PR

FINISHED
Internal note with the branch and the PR link; usage recorded; directory deleted

Three invariants that are structural, not instructions

INVARIANT

Never pushes to main

The push is always to a generated branch ref. The base branch is only ever read, and a protected base is refused outright.

INVARIANT

Never merges

The pull-request client has **no merge method**, no branch-delete method and no review-approval method. The capability is absent from the codebase, so apply mode could not merge even if the token allowed it — and the token is never given merge rights.

INVARIANT

Never fakes success

An empty diff, a failed validation command or an unavailable runtime all raise. The run is recorded as failed and an internal note names the step that stopped it.

Caps, and the asymmetry between them

TRIGGER	WHO STARTS IT	CAP	BYPASS
Manual	A staff member presses a button on the ticket	`100/day`	Administrators may exceed it, if configured
Automatic	A customer's ticket is created, if the board opts in	`20/day per board`	**Nobody. Ever.**

Nobody chose the automatic runs — a customer filing twenty tickets in an afternoon starts twenty runs, and on apply mode that is twenty clones and twenty pull requests. There is no bypass for the automatic cap **because there is nobody to grant one to**. "Today" is the board's own day, so a board configured for `Europe/Stockholm` rolls over at Stockholm midnight. Cancelled runs do not count, since they never started.

APPLY MODE IS PARTIALLY IMPLEMENTED AS SHIPPED

`AI_CODE_DRIVER` defaults to `unavailable`, and with that setting an apply run **fails immediately** with an internal note naming exactly what is missing. Nothing is faked and no empty pull request is opened. The working driver is `claude_code`, which runs the Claude Code CLI inside the isolated checkout and needs the binary (already installed in the Docker image) plus an API key in the worker's environment. Repository *cloning* is separately opt-in and defaults to off; with it off, suggest mode still runs and **records that the working tree was unavailable** rather than pretending it read the code.

Durability

Duplicate execution is guarded twice, because the two guards fail differently: a cache lock stops two workers starting the same run at the same moment, and an authoritative **status claim** flips `queued` to `running` in a single conditional update and checks the affected row count. Only transient provider failures are retried — a missing credential, an unavailable runtime, a protected branch or a failed test suite would fail

identically three times, so those stop on the first attempt rather than producing three identical internal notes twenty minutes apart.

A run is owned by the job that claimed it, so if that job disappears — a deploy, a restart, an out-of-memory kill — the ticket would show “Running” for ever. A scheduled command recovers stranded runs **every ten minutes**, and refuses to touch a run that still has a job on the queue.

> **A CUSTOMER HAS NO RELATIONSHIP WITH AI RUNS AT ALL**
>
> Not “cannot start one” — cannot see that they exist, cannot see a status, a cost or a failure. Every ability denies a customer as **404**, because a 403 on “AI runs for AQD-42” would confirm the feature is being used on their ticket.

08

Integrations

Three outward connections, each off until credentials exist, and each failing loudly rather than pretending.

GitHub

DIRECTION	WHAT IT DOES
Inbound	GitHub tells Nexora about branches, commits, pull requests and CI that mention a ticket key
Outbound	Apply mode pushes a branch and opens a draft pull request

A developer writes `NL-42` in a branch name or a commit message. GitHub posts a signed delivery; Nexora rate-limits it, verifies an HMAC-SHA256 digest over the **raw body** in constant time, records the delivery id under a unique index — the insert *is* the de-duplication claim — queues the work and answers 202 in well under GitHub's ten-second budget. A worker then resolves the reference and upserts a link on the ticket timeline.

> **THE SECURITY BOUNDARY IS THE REPOSITORY, NOT THE PATTERN**
>
> A webhook is an anonymous request whose only credential proves GitHub sent it — it does *not* prove who owns the repository it is about. So a delivery may only reach tickets on boards where an administrator has **already attached that repository**. The repository name in the payload is matched against the attachment table, and the resulting board ids are the *only* boards in scope. A repository nobody has configured resolves to nothing and the delivery is recorded as ignored.

EVENT	WHAT IS LINKED
`push`	Branch and commits. The **branch name is checked as well as each commit message**, because most teams put the key in the branch once rather than in every commit
`create`	Branch, only when the ref type is a branch — tags have no place on a ticket
`pull_request`	References taken from the title, the body and the head branch. Tracks opened, edited, closed, merged and ready-for-review
`check_suite`	CI status on an existing link
`status`	CI status on an existing link

Three link types and deliberately no more — a ticket's engineering trail is "somebody started a branch, commits landed on it, a pull request was opened"; anything further belongs in GitHub. Ticket events are written **only when something actually changed**, because GitHub sends a check suite for every push and an event per delivery would bury the ticket's own history within a day.

Slack

Configured **per board rather than per workspace**, because that is how the product is used: each customer's board belongs to a different delivery team with a different channel. The webhook URL therefore lives on the board, **encrypted at rest** — a Slack incoming-webhook URL is a bearer token wearing a URL's clothes, and Slack has no read-only variant.

EVENT	DEFAULT	FIRES WHEN
`customer_ticket_raised`	On	Somebody outside the delivery team files a new ticket
`customer_comment_posted`	On	A customer replies. **Internal notes are never announced**
`ticket_moved_to_done`	On	A ticket reaches a column marked terminal
`ai_run_finished`	Off	A run completes or fails. The analysis itself stays in the internal note

> WHY A SLACK MESSAGE IS DELIBERATELY THIN
>
> A Slack channel is **outside this application's authorization model**. Whoever is in the room reads the message, and Nexora has no idea which of them is a board member, which is a customer and which is a contractor added last week. So a message carries an identifier, a title, who did it, and a link — never a description, a comment body or an AI summary. **The link is the access control**: following it requires logging in, and the ordinary visibility rules then apply.

SMS

An SMS is the most intrusive notification this application can send: it reaches a person who is not at a computer, possibly asleep, and it costs money per message. So it has **exactly one trigger — a critical ticket being raised — and it is off until somebody deliberately turns it on.**

Recipients are **numbers rather than users**, on purpose: on-call is a rota, not a role, and the person carrying the phone this week is often not the person

whose account raised the board. A hard ceiling of five recipients applies whatever a board is configured with. Duplicates are suppressed by a unique index per ticket per number, plus a per-board cooldown — so if a script files forty critical tickets in a minute, the on-call phone rings once.

DRIVER	BEHAVIOUR
`unavailable`	**The default.** Every send is refused with a message naming exactly what to configure, and recorded as *failed*. No message is claimed to have been sent
`log`	Writes to the application log and records status *logged*, **never *sent*** — a dashboard counting these as delivered would be lying about whether an on-call engineer was reached
`46elks`	The real provider

Each record holds a phone number, which is personal data, so the table has a retention window of 90 days rather than growing for ever — long enough to answer “was the on-call rota reached about that incident?” without keeping it indefinitely. On the integrations screen the rota field is readable, because somebody has to be able to remove a colleague’s number when they leave it; the delivery *history* below it is masked.

09

Reporting and realtime

Two statistics screens, not one screen with a role switch

> **THE SEPARATION IS THE SAFETY PROPERTY**
>
> A shared class with a "for customer" flag would mean every future metric somebody adds is exposed by default and has to be remembered *not* to be — the wrong way round. Here the customer screen can only render what its own service offers, and that class offers six things. **There is no branch on either page that a careless edit could invert.**

SCREEN	WHO	METRICS
Team statistics	Staff only	Counts, by column, by priority, **by assignee**, **by label**, created by week, **visibility split**; flow metrics; AI usage and cost
Customer summary	**Everyone, staff included**	Counts, by priority, open versus closed, created by week, recent activity, recent tickets

The customer screen is open to staff deliberately: it is the only way for a team member to see exactly what a customer sees. When they open it they see *their own* visibility, not a simulated customer's, and the screen says so.

FLOW METRICS COME FROM HISTORY, NEVER FROM CURRENT STATE

"Twelve tickets are in Done" is **not throughput**: those twelve might have arrived over two years, and the eight finished last month and then reopened are invisible. Flow is a property of *transitions*, so these are read from the append-only event table — which carries the board and both column ids as dedicated indexed columns precisely so that "which tickets entered a done column on this board last week" is one index range scan.

Two limitations are surfaced to the reader rather than hidden: a ticket that has never left its column contributes no interval to the time-in-column figures, because counting "so far" would make a healthy board look worse the longer it stays healthy; and the fold is bounded, so a range wide enough to exceed it **reports that it was truncated** instead of quietly averaging a subset.

EXPORT

Dataset names are an **allow-list** — anything not in the list is refused rather than reflected, with no dynamic method call. Every dataset is derived from a scope resolved for the requesting viewer, so a CSV cannot contain a figure its requester could not see on screen. The date range is *corrected rather than trusted*: an unknown preset falls back to the default, reversed dates are swapped, and an absurd span is clamped to three years, so a hand-edited URL cannot ask the database for ten years of transitions. There is deliberately **no per-ticket export** — these are aggregates.

Realtime

CHANNEL	WHO MAY JOIN	CARRIES
`board.{id}.internal`	Staff members of that board	Content-free signals about internal changes
`board.{id}.customer`	Anyone who may view the board, customers included	Content-free signals about customer-visible changes
`users.{id}`	That user, while active	A nudge for the notification bell

> **EVERY CHANNEL IS PRIVATE — THERE IS NO PUBLIC OR PRESENCE CHANNEL**
>
> Because the internal channel is only signalled for internal changes, a customer's browser is **never even woken** by activity they are not allowed to know about — which closes the timing side-channel a single shared channel would leave open even with an empty payload. The rule for tickets is subtler than "is it customer-visible?": a ticket that has just been *hidden* also changes what customers see, so the customer audience is signalled when the ticket is customer-visible *now or was before the change*.

Events are dispatched after the surrounding transaction commits — broadcasting a change that then rolls back would tell every open board to re-read something that never happened. Realtime is optional: with broadcasting switched off everything still works, screens simply do not refresh by themselves.

Search, notifications and activity

CMD / CTRL + K

Command palette

Four categories — tickets, documentation, boards, people — at most five hits each, each filtered by the same visibility scopes the screens use. Pasting a ticket key resolves it as an indexed key seek and puts it at the top.

BELL

Notifications

Holds no data of its own. Everything is re-read on each render and re-checked against the visibility scopes, so what is shown is what the viewer may see *now*, not what they were allowed to see when it was written.

/ACTIVITY

Workspace activity feed

Ticket movement, assignments, comments, documentation changes and board configuration, newest first. **Staff only**, enforced three times — route middleware, a component check, and a SQL refusal — because every description is written for the delivery team.

APPEARANCE

Light, dark, system

Three states rather than a boolean, because “follow the operating system” is a genuine third answer. The preference is stored on the user row so it follows a person between devices, cached in local storage, and applied by an inline script before the bundle downloads so there is no flash of the wrong theme.

SEARCH IS PLAIN SQL, ON PURPOSE

There is no Scout, Meilisearch, Algolia or Elasticsearch. An external index has no idea that a ticket is internal, so the filtering would have to be reproduced in the engine’s query language, correctly, for ever — and it would hold a second copy of every ticket description outside this application’s visibility rules. The documented next step, when `LIKE` stops scaling, is a **MySQL FULLTEXT index**, which stays inside MySQL and needs no new service. Scout is the step after that.

10

Architecture

A single Laravel application rendered server-side, made interactive by Livewire, and deployed as one Docker image run three ways.

Technology stack

LAYER	TECHNOLOGY	VERSION	WHY IT IS HERE
Language	PHP	`^8.2 (8.3 in the container)`	—
Framework	Laravel	`^12.0`	Policies, queues, broadcasting, notifications and Eloquent scopes are the actual enforcement mechanisms, not conveniences layered on top
Interactive UI	Livewire	`^4.4`	Server-rendered interactivity without a second application. One component owns a whole board, so a drag-and-drop move re-renders source column, destination column and both counts in one round trip
Styling	Tailwind CSS	`^4.0`	Utility styling with no component stylesheet to fight
Editor	TipTap / ProseMirror	`^3.31`	A schema-constrained editor: the document can only ever contain nodes the schema declares, which is a security property as much as a correctness one
Realtime	Laravel Reverb + Echo	`^1.11`	First-party WebSocket server, so realtime needs no third-party service and channel authorization runs through Laravel's own gates
Database	MySQL	—	Relational integrity is load-bearing — foreign keys and the unique `(board_id, number)` key are what make the rules hold under concurrency
Queue	Database, Redis optional	—	No Redis required for a working deployment; Redis is needed only for multi-replica scheduler locks and Reverb scaling
AI	Anthropic SDK; OpenAI over HTTP	`^0.45`	Two providers with independent field inheritance
Storage	S3-compatible via Flysystem	`^3.35`	Container filesystems are replaced on every deploy; a production boot with an ephemeral attachment disk is refused outright
Runtime	nginx + PHP-FPM, Supervisor, Alpine	—	One image with three start commands, so the worker image cannot drift from the web image
Hosting	Railway	—	Declarative per-service definitions; TLS terminated at the edge

Layers, and where the rules live

LAYER	RESPONSIBILITY	MAY IT DECIDE ACCESS?
`routes/`	URL to component, coarse role middleware	First line only
`Http/Middleware`	Role gate, active-account gate	Coarse only
`Livewire`	Screen state and interaction	Calls policies; holds no rules
`Policies`	Per-record authorization	**Yes — single-record rules**
`Services`	Readers, scopes, integrations	**Yes — SQL-level scoping**
`Actions`	One write each, with its own rules	**Yes — field-level rules**
`Models`	Relationships and query scopes	**Yes — visibleTo() scopes**
`Support`	Value objects, formatting, parsing	No

Request lifecycle

PROXY
Trust proxies — Railway terminates TLS, so recover the real scheme and client IP

WEB GROUP
Session, cookies, CSRF

SESSION BIND
The session is bound to the password hash it was created with

ACTIVE CHECK
A deactivated account is logged out on its very next request

ROLE GATE
Where present; plus password confirmation on administration screens

MOUNT
Policy check — denying as 404 for anything a person must not know exists

RENDER
Reader service → Eloquent scope → SQL with the viewer applied

OUTPUT
Markdown rendered, sanitised, with viewer-aware ticket links

EVERY LIVEWIRE INTERACTION IS A NEW HTTP REQUEST

Components re-authorize on mount, on every action and on every render — they never trust what the previous render decided. This is why the documentation screen re-resolves its page through the finder on each render: Livewire rehydrates models by primary key, which bypasses query scopes.

There is no workspace entity

Worth stating plainly, because it shapes everything else: **there is no workspace table and no tenant model.** One deployment is one workspace.

The unit of access is the board, and almost everything that looks like a workspace-level setting is in fact a board setting. The two genuine exceptions are the global AI settings — a single row — and user administration.

11

Security

Authentication

CONTROL	VALUE
Method	Laravel session-based authentication, screens rebuilt as Livewire components
Password policy	At least **12 characters** with letters and numbers everywhere; in production additionally mixed case and **checked against known breach corpora**
Hashing	bcrypt at 12 rounds; the attribute is cast `hashed`, so a plaintext value can never be stored in the clear
Login throttle	5 failed attempts, keyed on **email *and* client IP** — keying on the email alone would let one attacker lock every account out from a single address
Deactivated accounts	Refused **even with correct credentials**; the session is established then immediately invalidated
Session fixation	The session id is rotated on successful login
Password change	**Invalidates every other session** for that account
Re-authentication	Required again inside a valid session for global AI settings, user administration, and board creation or editing
Registration	404 unless explicitly enabled; a self-registered account gets the least-privileged role

NOT IMPLEMENTED

Two-factor authentication, SSO / SAML sign-in, and email verification (the column exists, the flow does not). If any of these is a requirement for a customer, it is new work.

Secret handling

ENFORCED BY CONSTRUCTION, AND ASSERTED BY A DEDICATED TEST

No credential in this system is ever rendered back to a browser, written to a log line, or included in an exception message.

SECRET	STORED	WRITE-ONLY IN THE UI?
Anthropic / OpenAI keys	Environment, *or* encrypted workspace row, *or* encrypted per board — read by one vault class and nothing else	**Yes**
Slack webhook URL	**Encrypted** in the board settings JSON	**Yes** — replaced, never read back
GitHub token and webhook secret	Environment only	n/a
SMS provider credentials	Environment only	n/a

Credential screens render *presence* and the last four characters, never the value; the vault has no method that would return a key to a component. Somebody who gains access to those screens can **replace** a credential — which is visible in the rotation timestamp and the updating user — but cannot read one out. Decryption failure degrades to "not configured" rather than throwing, so rotating the application key makes a board stop posting to Slack instead of breaking every ticket save.

The GitHub token is **deliberately weak**: it needs exactly `contents: write` to push a branch and `pull_requests: write` to open one. It is never given merge rights, and the client has no merge method, so a future phase that genuinely needs to merge has to add the capability and be reviewed for it.

Input and output

RENDERING

One pipeline for all user prose

Markdown to HTML with raw HTML stripped and unsafe link schemes rejected, then mentions highlighted and ticket references linked — **but only for tickets the particular reader may open**. Because steps two and three are viewer-dependent, nothing in this pipeline may be cached across viewers.

EDITOR

An allow-list in both directions

Browser to server, the posted document is attacker-controlled, so it is cleaned before the Markdown converter sees it — nothing hostile is stored, not even as inert text that would still surface in search results, board cards and AI prompts. An allow-list, because a deny-list has to anticipate whatever parser-confusion trick lands in browsers next year.

MODEL OUTPUT

Never rendered directly

Charts, tables and KPI rows are validated into typed value objects before anything is drawn. There is no charting library receiving model-authored configuration, so there is no option in such a library that turns a string into a callback.

UPLOADS

Two ceilings, two allow-lists

Record attachments cap at 10 MB and 20 per owner; AI conversation files at 20 MB and 10 per conversation, and must pass **both** an extension allow-list and the detected MIME type. Stored paths are generated, never derived from the uploaded filename.

Two absences in the AI allow-list are deliberate: **SVG**, because it is a document that can carry script rather than a picture, and here an "image" is decoded and sent to a vision model; and **ZIP**, a container whose contents nothing has validated. Downloads never link straight at stored bytes — every request is authorized against the owning record first, then redirected to a signed URL valid for five minutes.

Webhook signature verification

PROPERTY	IMPLEMENTATION
Fail closed with no secret	With the secret unset, *every* delivery is rejected. There is no "skip verification when unconfigured" path
Verify the raw body	Never the parsed or re-encoded input — key order, unicode escaping and float formatting all differ after a round trip
Constant-time comparison	So the number of correct leading bytes cannot be walked out through timing
SHA-256 only	The legacy SHA-1 header is ignored. Accepting it would make the endpoint's real strength SHA-1's, because the attacker picks the header

Failures are deliberately indistinguishable: a missing secret, a missing header and a wrong digest all return the same `401 Invalid signature.` This is the only CSRF-exempt path in the application, and the exemption is replaced by something stronger — an HMAC signature proves the request came from somebody holding the shared secret, whereas CSRF only proves a request came from our own page.

Infrastructure controls

CONTROL	STATUS
Storage durability enforced at boot — production refuses to start on a non-durable attachment disk, and *stats the mount* for a volume rather than trusting the name	In place
`X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, server tokens off	In place
HTTPS forced for generated URLs in production — without it, password-reset links would be `http://`	In place
Rate limiting on the webhook endpoint (300/min per IP) and the AI voice endpoints (30/min per user)	In place
Content-Security-Policy header	Not set
Strict-Transport-Security header	Not set
Rate limit on the assistant's *text* endpoint — it is authenticated and budget-bounded, but a request limiter would still be worth adding	Not set

TWO DEPLOYMENT SETTINGS TO CHECK BEFORE GO-LIVE

`SESSION_SECURE_COOKIE` ships as `false`, which is correct for local HTTP development and **wrong for production**. And the demo seeder contains passwords in source, documented in the class as a deliberate demo trade-off — those accounts must be changed before anything real lands.

12

Data model

MySQL in production, SQLite in memory for the test suite. 41 migrations, 25 models.

The tables that carry the rules

TABLE	WHY IT MATTERS
`board_members`	**This table is the access-control list.** Unique on `(board_id, user_id)`, with an audit column recording who granted access. Administrators are not implicit members — membership is a factual question, and the administrator bypass is an authorization one
`tickets`	Unique on `(board_id, number)`. The index on `(board_id, customer_visible)` is what makes the customer boundary cheap: the filter applied to almost every ticket read is a composite index seek rather than a scan
`comments`	Indexed on `(ticket_id, stream, created_at)`, leading with the scoping column so a stream-filtered read is an index range
`doc_pages`	Self-referencing parent, restricted on delete, slug unique per board
`ticket_events`	**Append-only.** Carries the board and both column ids as dedicated indexed columns rather than only inside the JSON payload, precisely so flow metrics are one index range scan. A later migration backfilled them
`attachments`	Polymorphic with an **enforced** morph map, so an attachment pointing at a type nobody registered fails loudly instead of resolving to something unchecked
`webhook_deliveries`	Unique on `(source, delivery_id)` — the insert *is* the de-duplication claim, atomic across containers in a way that select-then-insert would not be
`sms_messages`	Unique on `(event, ticket_id, recipient)` — duplicate suppression enforced by the database rather than a cache, because the guarantee has to survive a cache flush and a race between two workers
`ai_tool_invocations`	The AI audit log: one row per tool call, successful or refused

Referential integrity strategy

PATTERN	USED FOR	EFFECT
`cascadeOnDelete`	Board id on almost everything; ticket id on children	Deleting a board removes its entire graph
`restrictOnDelete`	A ticket's column; a page's parent	Prevents orphaning structure
`nullOnDelete`	Every authorship column and the assignee	**Deleting a user preserves the history they created** — a ticket keeps existing with no creator rather than vanishing

READ A ROLLBACK BEFORE RUNNING ONE

Several `down()` methods are destructive by necessity — restoring a `NOT NULL` constraint requires removing the rows that violate it. The migration that allowed workspace-scoped AI conversations deletes those rows on the way back down.

13

Deployment and operations

One Docker image, built in three stages, run as three Railway services with three different start commands — so the worker image cannot drift from the web image.

SERVICE

web

Supervisor running PHP-FPM, nginx, and a queue worker dedicated to the `attachments` queue. Health-checked at `/health`.

SERVICE

worker

`queue:work --queue=ai,default`, three tries, restarted always. The `ai` queue is named explicitly and listed first — a bare `queue:work` listens only to `default` and would leave every AI run queued for ever.

SERVICE

reverb

The WebSocket server, on its own public domain. **One replica only** — Reverb holds its connections in the memory of the process that accepted them, so a second replica would only reach half the clients.

The three queues

QUEUE	CARRIES	RUNS IN
`ai`	Ticket runs	worker — a run takes minutes, and must never delay a broadcast
`default`	Broadcasts, in-app notifications, GitHub webhook processing, Slack, SMS	worker — millisecond work
`attachments`	AI file processing	**The web container** — see below

WHY ATTACHMENT PROCESSING RUNS IN THE WEB CONTAINER

The reason is the disk rather than the workload. The job opens the file that was just uploaded, and on a deployment whose attachment disk is a Railway volume, a volume attaches to **exactly one service** — so the upload request writes the file in the web container and a worker in the other container asks its own mount and finds nothing. Worse, a missing object is a *legitimate* condition in the reader, so the row settles into `failed` with no exception and nothing in the log to chase. An S3-backed deployment has no such constraint and can consolidate the queues.

THE SINGLE MOST COMMON MISCONFIGURATION

Without a queue worker, several features silently do nothing — and nothing errors, because all of it is queued by design. Realtime updates never arrive, no Slack message is posted, no SMS is sent, no pull request is ever linked to a ticket, and every AI run stays queued for ever.

Scheduled housekeeping

SCHEDULE	COMMAND	PURPOSE
Every 10 minutes	`ai:reap-stalled`	Recover runs stranded by a worker that died mid-run
Daily 03:15	`workspace:prune`	Webhook deliveries past 30 days; SMS records past 90 days, in chunks rather than one unbounded delete
Weekly	`queue:prune-failed`	Keep the failed-jobs table readable — 168 hours, long enough to investigate a Monday failure on the following Friday
Daily	`auth:clear-resets`	Expired password-reset tokens

Everything scheduled is housekeeping. Nothing a user waits for is scheduled, and nothing here is required for the application to function — a deployment with no scheduler works, it simply accumulates rows. The exception worth watching is the stalled-run reaper. Failed jobs are **not** retried automatically, because blindly replaying a batch would re-post internal notes and re-send messages.

Build-time versus run-time configuration

THE REALTIME CLIENT KEYS ARE BUILD ARGUMENTS, NOT RUNTIME VARIABLES

Vite inlines them at build time, and the client only constructs its connection when a key is present. **An image built without one ships a browser bundle that never subscribes to anything**, whatever the server is configured to broadcast — so changing one requires a rebuild of the web image, not a restart. Only the *public* key belongs in that stage; the app secret must never be passed there, because everything in it is readable in the shipped JavaScript.

The application is configured by 180 environment variables. The entrypoint aborts if the application key is unset, caches configuration, routes, views and events at boot, and optionally runs migrations with an isolation lock.

14

Quality assurance

1,718	6,436	28	311s	3
Tests passing	Assertions	Security test files	Full suite duration	Failing — all test defects

Three suites

SUITE	FILES	PURPOSE
Unit	16	Pure logic: enums, value objects, parsers, cost calculation
Feature	90	End-to-end behaviour through routes and Livewire components
Security	28	**Authorization and isolation, runnable alone as a fast CI gate**

The security suite asserts the customer boundary directly: that internal tickets are invisible and unwritable, that internal notes and their *count* are hidden, that the documentation ancestor rule holds, that non-members get 404 rather than 403, that aggregates and CSVs cannot leak internal figures, that the palette cannot surface unreachable rows, and that no credential reaches a response, a log line or an exception message. On the AI side it asserts conversation ownership, that a customer's assistant is read-only with staff tools withheld, that proposals are authorized against the confirming person per field, and that charts cannot aggregate hidden data. The suite **never calls a real provider** — a fake provider scripts responses, and a second one asserts that a code path does *not* reach a provider at all.

The three failures

ALL THREE ARE TEST DEFECTS, NOT APPLICATION DEFECTS

Two are pre-existing theme-rendering assertions. The third reads at a glance like a customer-isolation failure and is not: the test creates 2 customer-visible and 5 internal tickets, and **the two assertions that matter both pass** — the customer's answer contains `(total 2)`, and the staff answer contains `(total 7)`. The failing assertion is a belt-and-braces check that the digit `7` appears nowhere in the customer's answer, so the hidden count could not be recovered by subtraction — but the report header carries today's date, and `2026-09-17` contains a 7. It therefore fails on roughly a third of calendar days. The fix is to narrow the assertion to `(total 7)`, not to change any application behaviour.

Gaps

GAP	NOTE
Browser / end-to-end tests	**None.** Drag-and-drop, the editor, the command palette and theme switching are tested only at the server boundary
JavaScript unit tests	**None.** The client modules are untested in isolation
CI pipeline	**Not present in the repository.** A 139-file suite with a dedicated fast security gate exists, but nothing runs it automatically
Coverage thresholds, load tests, mutation testing	Coverage is configured but no threshold is enforced

15

Implementation status

The section to read before making a commitment to anybody about what Nexora does.

Partially implemented

CAPABILITY	WHAT EXISTS	WHAT IS MISSING
Apply-mode code generation	The whole pipeline: clone, branch, validate, commit, push, draft PR, with every refusal point	The driver **defaults to unavailable**. Needs the `claude_code` driver plus a key in the worker's environment
Repository cloning	Git client, checkout, context summarisation	**Defaults to off.** Suggest mode then reasons from metadata and the ticket alone, and records that the working tree was unavailable
External knowledge lookup	An interface, a tool, configuration, and a refusing default	**No driver ships.** Outside Project mode still works; what is absent is live third-party lookup. Adding one is a class plus a match arm
OpenAI model catalogue	A working adapter and provider selection	Context windows and pricing are not stated for OpenAI models, so they render blank and report no cost. Vision is opt-in per id
SMS delivery receipts	The callback URL may be configured	**No receiving endpoint is exposed.** Setting it would produce retries at the provider
Email	Password reset works	No other transactional mail, and no email notification channel — notifications are in-app, Slack or SMS
Board archiving	The column and the query scope exist	No user interface to archive or restore a board
Ticket type	Stored, filterable, in the AI schemas	No workflow behaviour differs between a bug and a feature

Not implemented

CAPABILITY	STATUS
AI Standup Generator	**Verified absent by a repository-wide search** — the only match is a test fixture filename. There is no standup service, command, component, route, view or job. The building blocks exist (activity and code-activity readers), so it is buildable, but nothing generates a standup today
General REST / JSON API for third parties	No general API and no specification. The one machine-facing interface is the remote MCP server at `/mcp`, which Claude and ChatGPT reach with an OAuth token on behalf of one signed-in person, to read and to make the changes that person could make by hand. Other integration is inbound webhook plus outbound HTTP client
Outbound webhooks from Nexora	Only Slack and SMS leave the application
Two-factor authentication, SSO, email verification	Not implemented
Documentation version history, page comments, collaborative editing, PDF export	Not implemented
Time tracking	An estimate is stored; nothing logs actual time
Sprints, iterations, epics, recurring tickets, templates, custom fields	Not implemented
Ticket import / export	Not implemented. Per-ticket CSV export is deliberately absent — the exports are aggregates
Multi-tenant organisations	**No workspace entity.** One deployment is one workspace; the identifier service notes the two closures that would need to become organisation-aware
Browser tests, JavaScript tests, CI configuration	Not present
Content-Security-Policy and HSTS headers	Not set

16

Findings and next steps

Twelve items found while compiling this document. None is a functional defect in itself, but each is worth an owner. They are ordered by how cheaply they can be closed against how much risk they carry.

#	FINDING	SUGGESTED ACTION
01	**Product naming is inconsistent.** The runtime workspace name defaults to *Aqueduct Workspace* and the package is `aqueduct/workspace`, but the AI system prompt, source comments and one accessibility label say *Nexora*. A deployment that has not set the name presents as Aqueduct while its assistant calls itself Nexora	Decide the product name and set `WORKSPACE_NAME` and `WORKSPACE_SHORT_NAME` accordingly
02	**No version identifier anywhere.** No git tag, no version field, no application constant	Adopt tags, and surface a version in `/health`. Support and rollback currently have no vocabulary
03	**No CI configuration in the repository.** A 139-file suite with a dedicated fast security gate exists, but nothing runs it automatically	Run the security suite on every push and the full suite on merge. This is the single highest-value item on the list
04	**A date-sensitive assertion in the security suite** fails on roughly a third of calendar days, for no real reason — and it fails in the suite people trust most	Narrow the assertion to `(total 7)` rather than the bare digit
05	**Two pre-existing theme test failures.** Not a regression, but two permanently red tests	Fix or quarantine, so that “green” means green
06	`README.md` **is empty.** Two source files refer readers to “the deployment notes in README.md”, which do not exist	Populate it, or point it at this document
07	**The demo seeder contains passwords in source**, documented as a deliberate demo trade-off	Change those accounts before anything real lands, as the class itself instructs
08	**No Content-Security-Policy or HSTS header.** Two cheap defence-in-depth controls	Add both at nginx. HSTS in particular is near-free on a platform that already terminates TLS
09	`SESSION_SECURE_COOKIE` **ships as false.** Correct for local HTTP, wrong for production	Add it to a deployment checklist
10	**The default capability mode disagrees with itself** — configuration says `agent`, the database column default says `observer`. Both are deliberate and documented, but a deployment’s effective posture depends on whether a settings row exists	Decide one, or explain the interaction on the admin screen. A fresh deployment wanting the cautious posture should set the mode explicitly
11	**Ticket type has no behaviour.** Stored and filterable, but nothing differs between a bug and a feature	Either give it behaviour or document it as a label
12	**Board archiving has no interface.** The column and scope exist and nothing can reach them	Add the control, or remove the column

WHAT NONE OF THESE ARE

None of the twelve is a breach of the customer boundary, a data-loss risk or a functional regression. The boundary is enforced in four overlapping layers and asserted by 28 dedicated security test files; the two most consequential items on this list are operational — **no CI pipeline** and **no version identifier** — and both are a day's work.

17

Glossary

The terms this product uses, in the sense it uses them.

Board
The unit of work, access and configuration. Everything hangs off one, and there is no entity above it.

Board member
An explicit row linking a person to a board. **This is the access grant**; administrators bypass it.

Ticket key
The human identifier — the board's prefix, a hyphen, the per-board number, as in `NL-42`. What GitHub branches and commits reference.

Customer
The security boundary of the product. May never observe internal content, on any board, including boards they belong to.

Staff
Administrators and team members together — the set that may see internal content.

Customer-visible
The positive visibility flag on a ticket or page. Defaults to false, so anything created by code that forgets to set it is internal.

Internal note
A comment in the internal stream. Customers cannot read it, count it, or detect that it exists.

Stream
Which conversation a comment belongs to: customer or internal.

Capability mode
The administrator's ceiling on what the AI may ever attempt: Observer, Operator or Agent.

AI Observer
The most restrictive mode. Reads and answers; writes nothing, not even an internal note — so no ticket run may start.

AI Operator
The middle mode. Proposes changes a person confirms, and permits manual suggest runs.

AI Agent
The most permissive mode. Adds unattended work: automatic runs, apply-mode pull requests, and reversible changes carried out without a confirmation step.

Chat mode
The asker's choice per conversation: Reading, Writing or Everything.

Knowledge scope
Where an answer may be sourced from: Project only (the default) or Outside Project, which adds general knowledge alongside the project and widens data access by nothing.

Proposal
A change the AI has *described*. A preview until a person confirms it — at which point it is re-resolved, re-authorized and executed through the ordinary action.

Run
One AI working session on one ticket. Suggest (posts an internal note) or apply (opens a draft pull request).

Tool
A read-only lookup the AI may call. Runs with the asking person as the viewer, and is audited whatever the outcome.

Draft (documentation)
The autosave buffer for a *published* page. Nothing renders it to a reader.

Slug

The URL identifier for a board or page. Primary keys are never exposed.

Workspace

The whole deployment. Not an entity — there is no workspace table.

Provenance. Compiled by reading the repository at commit `30bfc83`: all 19 configuration files, every route, all 25 models, all 41 migrations, all 8 policies, all 27 enums, the key actions and services across AI, GitHub, Slack, SMS, statistics, search and attachments, all 35 Livewire components, the Blade and JavaScript trees, the container and platform configuration, and the test inventory.

The full test suite was executed during compilation — 1,718 passed, 3 failed, 6,436 assertions in 311 seconds — and each failure was reproduced and diagnosed individually. Every permission claim was derived from middleware, policies, actions, query scopes or a tool's own availability method, never assumed. One claim in an early draft, that login was not rate limited, was found to be wrong on verification and corrected. No application code was modified in the production of this document.

Nexora — The Next Way to Work Together · Complete System Documentation · 17 September 2026 · Proprietary and confidential